

Ch: 2 - Quality Assurance Framework

What Is Quality?

In the context of software testing, **quality** refers to how well a software product meets the requirements and expectations of its users, as well as how reliably it performs under various conditions.

Quality means “meeting requirements.”

Reliability, Availability, and Maintainability (RAM)

Prevention Vs. Detection

Prevention:

Prevention focuses on avoiding defects before they occur.

For example, when new functionality is added to handle product searches, automated tests check if search results display correctly.

Adding any new component to system does not cause any problems in existing system(example : Adding new components like fasttag, mobile recharge, paying electricity bill in amazon does not affect its basic functions)

Prevent quality defects or deficiencies in the first place, and to make the products assessable by quality assurance measures.

Detection

Detection, on the other hand, focuses on finding defects after they've been introduced.

Real-time example of detection: After the online banking app is built, the QA team runs a series of test cases, including boundary tests (e.g., testing the app with large transaction amounts) and security tests (e.g., checking for vulnerabilities like SQL injection).

During testing, they find an issue where an incorrect validation allows users to transfer more money than they should. The team then reports this bug, and the developers fix it.

During testing, the tester applies a valid discount code, and the final price isn't reflecting the discount correctly

One tester tries entering a blank password, and the system allows it, which wasn't intended.

Verification versus Validation

Verification is proving that a product meets the requirements specified during previous activities carried out correctly throughout the development life cycle.

Validation confirms that the system meets the customer's requirements at the end of the life cycle. It is a proof that the product meets the expectations of the users, and it ensures that the executable system performs as specified.

Verification ensures that the software functions as intended and has the required attributes (e.g., portability), and increases the chances that the software will contain few errors (i.e., an acceptable number in the final product).

Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is the process of ensuring that software is built correctly and meets defined standards

It is the systematic activities providing evidence of the fitness for use of the total software product. Software quality assurance is achieved through the use of established guidelines for quality control to ensure the integrity and long life of software.

Quality Assurance

Example : Test Automation for E-Commerce Website

Scenario: An e-commerce website has a feature where users can add items to their cart, proceed to checkout, and make a purchase.

QA Process: The QA team writes automated test scripts to test the core functionality of the website. Every time a new feature is added (like a new payment option), automated tests are run to make sure the checkout process still works across all browsers and devices. These tests ensure that features work as expected without breaking existing functionality.

Components of Quality Assurance

Software Testing and Continuous Quality Improvement



Software testing is a popular risk management strategy. It is used to verify that functional requirements were met

The major purpose of verification and validation activities is to ensure that software design, code, and documentation meet all the requirements imposed on them.

Examples of requirements include user requirements; specifications derived from and designed to meet user requirements; code review and inspection criteria; test requirements at the modular, subsystem, and integrated software levels; and acceptance testing of the code after it has been fully integrated with hardware.

Quality control is defined as the processes and methods used to monitor work and observe whether requirements are met. It focuses on reviews and removal of defects before shipment of products. Quality control should be the responsibility of the organizational unit producing the product.

Quality control consists of well-defined checks on a product that are specified in the product quality assurance plan. For software products, quality control typically includes specification reviews, inspections of code and documents, and checks for user deliverables.

Quality control is designed to detect and correct defects, whereas quality assurance is oriented toward preventing them.

QC can be done through :

1. personnel's peer group or the department's software quality coordinator can inspect the documents.
2. A configuration control board may be responsible for quality control.
3. Inspections are traditional functions of quality control, that is, independent examinations to assess compliance with some stated criteria. Peers and subject matter experts review specifications and engineering work products to identify defects and suggest improvements.

Software configuration management is concerned with labeling, tracking, and controlling changes in the software elements of a system.

It controls the evolution of a software system by managing versions of its software components and their relationships.

The purpose of software configuration management is to identify all the interrelated components of software and to control their evolution throughout the various life-cycle phases.

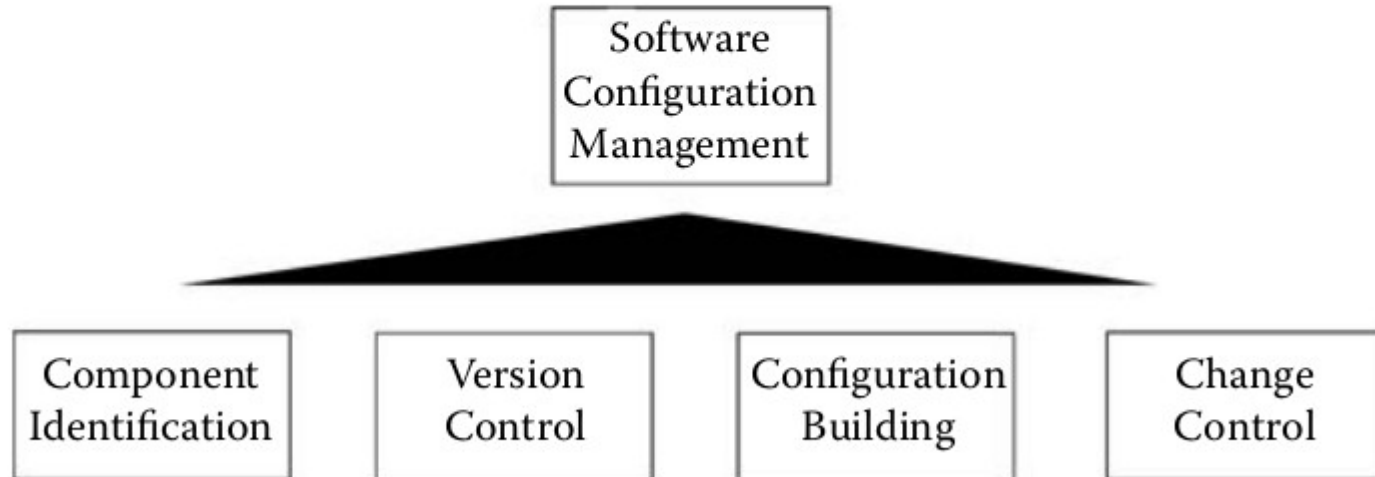
Software configuration management is a discipline that can be applied to activities including software development, document control, problem tracking, change control, and maintenance. I

Software configuration management consists of activities that ensure that

design and code are defined and cannot be changed without a review of the effect of the change itself and its documentation. The purpose of configuration management is to control code and its associated documentation so that final code and its description are consistent and represent those items that were actually reviewed and tested.

Elements of Software Configuration Management

Quality Assurance Framework



A basic software configuration management activity is the identification of the software components that make up a deliverable at each point of its development.

Software configuration management provides guidelines to identify and name software baselines, software components, and software configurations.

A software configuration is a collection of software elements that comprise a major business function.

An example of a configuration is the set of program modules for an order system

Version Control:

Software is frequently changed as it evolves through a succession of temporary states called versions. A software configuration management facility for controlling versions is a software configuration management repository or library.

Version control provides the tractability or history of each software change, including who did what, why, and when.

Configuration Building :

To build a software configuration, one needs to identify the correct component versions and execute the component build procedures. This is often called configuration building.

Change Control:

Change control is the process by which a modification to a software component is proposed, evaluated, approved or rejected, scheduled, and tracked.

Its basic foundation is a change control process, a component status reporting process, and an auditing process.

Software Quality Assurance Plan

The software quality assurance (SQA) plan is an outline of quality measures to ensure quality levels within a software development effort.

The plan is used as a baseline to compare the actual levels of quality during development with the planned levels of quality.

Steps to Develop and Implement a Software Quality Assurance Plan

Step 1: Document the Plan

Purpose Section

Reference Document Section

Management Section

Documentation Section

Standards, Practices, Conventions, and Metrics Section

Reviews and Inspections Section

Software Configuration Management Section

Problem Reporting and Corrective Action Section

Tools, Techniques, and Methodologies Section

Code Control Section

Media Control Section

Supplier Control Section

Records Collection, Maintenance, and Retention Section

Testing Methodology

Obtain Management Acceptance:

Management participation is necessary for the successful implementation of an SQA plan.

Management is responsible for both ensuring the quality of a software project and for providing the resources needed for software development.

Step 3: Obtain Development Acceptance

No SQA plan is successfully implemented without the involvement of the software team members and their managers in the development of the plan.

Because project teams generally have only a few members, all team members should actively participate in writing the SQA plan

Step 4: Plan for Implementation of the SQA Plan

The process of planning, formulating, and drafting an SQA plan requires staff and word-processing resources. The individual responsible for implementing an SQA plan must have access to these resources.

Step 5: Execute the SQA Plan:

The actual process of executing an SQA plan by the software development and maintenance team involves determining necessary audit points for monitoring it.

Quality Standards

ISO9000

Sarbanes–Oxley

The Sarbanes–Oxley Act is designed to ensure the following within a business:

- > There are sufficient controls to prevent fraud, misuse, or loss of financial data/ transactions. In many companies, most of these controls are IT-based.
- > There are controls to enable speedy detection if and when such problems occur.
- > Effective action is taken to limit the effects of such problems.

Testing activities in SQA involve:

- Test planning and strategy

- Test case design and execution

- Bug reporting and tracking

- Continuous integration and testing

Example: The team tests the search functionality to ensure users can search for products correctly by product name, category, or price range

Example: The team tests the checkout page to ensure that all fields are visible, align correctly, and are easy to navigate. They also test if buttons like "Proceed to Payment" are easily clickable on mobile devices.

The team performs load testing by simulating 10,000 users accessing the website simultaneously to check if it slows down or crashes.

The team tests the login page for potential SQL injection vulnerabilities by inputting malicious SQL queries in the username and password fields. (where an attacker can bypass login validation by entering certain SQL commands.)

Example: The team tests the product detail page in different browsers (Chrome, Firefox, Safari, Edge) to check for layout or functional discrepancies.

Example: After fixing the issue with the payment gateway, the team retests the entire checkout flow to ensure that adding items to the cart, applying discount codes, and completing the payment are still functioning correctly.

Software Configuration Management (SCM)

SCM ensures that the software development process remains organized, traceable, and that the final product is consistent and of high quality.

The SQA team may track the status of various components, such as whether a certain feature module is in development, tested, or has been approved for release.

Capability Maturity Model (CMM)

In software testing, CMM provides a structured approach for improving testing processes in order to achieve higher quality and more reliable software products

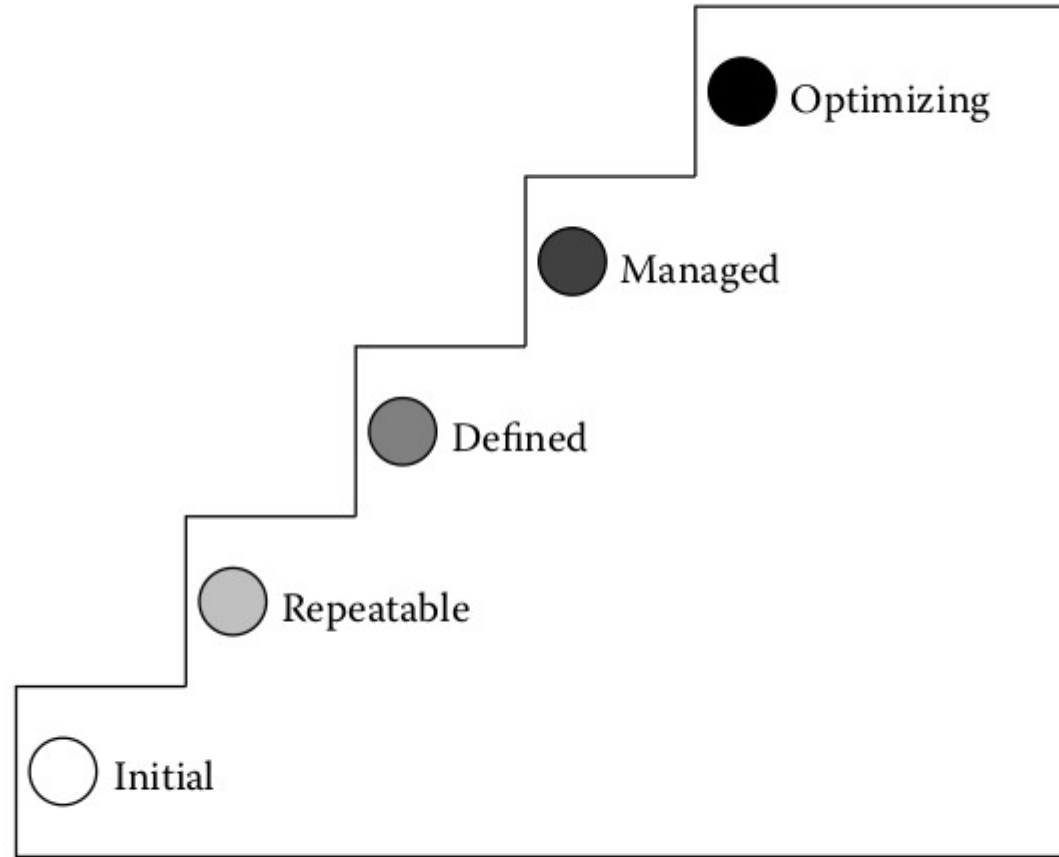


Figure 2.3 Maturity levels.

The Software Engineering Institute–Capability Maturity Model (SEI–CMM) is a model for judging the maturity of the software processes of an organization and for identifying the key practices that are required to increase the maturity of these processes.

The CMM is organized into five maturity levels

1. Initial. The software process is characterized as ad hoc, and occasionally even chaotic. Few processes are defined, and success depends on individual effort and heroics.
2. Repeatable. Basic project management processes are established to track cost, schedule, and functionality. The necessary process discipline is in place to repeat earlier successes on projects with similar applications.
3. Defined. The software process for both management and engineering activities is documented, standardized, and integrated into a standard software process for the organization. All projects use an approved, tailored version of the organization's standard software process for developing and maintaining software.
4. Managed. Detailed measures of the software process and product quality are collected. Both the software process and products are quantitatively understood and controlled.
5. Optimizing. Continuous process improvement is enabled by quantitative feed-back from the process and from piloting innovative ideas and technologies.

CMMI

The CMMI Product Suite provides the latest best practices for product and service development and maintenance

Malcolm Baldrige National Quality Award



1. Approach: Approach indicates the method that the company uses to achieve the purposes. The following are the factors to decide on the correct approach:
the degree to which the approach is prevention-based; the appropriateness of the tools, techniques, and methods; the effectiveness of their use; whether the approach is systematic, integrated, and consistently applied; effective selfevaluation and feedback; quantitative information gathered; and the uniqueness and innovativeness of the approach.
2. Deployment: This concerns the areas where the approach is deployed. It evaluates whether the approach is implemented in all the products and services and all internal processes, activities, facilities, and employees.
3. Results: This refers to the outcome of the approach. The quality levels demonstrated, rate of quality improvement, breadth, significance, and comparison of the quality improvement and the extent to which quality improvement is demonstrated are the key factors involved.

Baldrige Performance Framework

1. *Leadership*—Examines how senior executives guide the organization and how the organization addresses its responsibilities to the public and practices good citizenship. Evaluations are based on the appropriateness, effectiveness, and extent of the leader's and the company's involvement in relation to the size and type of business.
2. *Measurement, analysis, and knowledge management*—Examines the management, effective use, analysis, and improvement of data and information to support key organization processes and the organization's performance management system. The scope, management, and analysis of data depend on the type of business, its resources, and the geographical distribution.
3. *Strategic planning*—Examines how the organization sets strategic directions and how it determines key action plans. Evaluations are based on the thoroughness and effectiveness of the processes.

4. *Human resource focus*—Examines how the organization enables its workforce to develop its full potential and how the workforce is aligned with the organization's objectives. Evaluation depends on the human resource approach of the company.

5. *Process management*—Examines aspects of how key production/delivery and support processes are designed, managed, and improved. The types of products and services, customer and government requirements, regulatory requirements, and number of business locations are the factors influencing this.

6. *Business results*—Examines the organization's performance and improvement in its key business areas: customer satisfaction, financial and marketplace performance, human resources, supplier and partner performance, operational performance, and governance and social responsibility. The category also examines how the organization performs relative to competitors.

7. *Customer and market focus*—Examines how the organization determines requirements and expectations of customers and markets; builds relationships with customers; and acquires, satisfies, and retains customers.